

Lab 2: Controller Design

MTE 484: Digital Control Systems
Instructor Sergio Livera

Department of Mechatronics and Mechanical Engineering

Levi Rogalla
Alex Veric

November 8, 2026

We acknowledge and promise that:

(a) We are the sole authors of this lab report and associated simulation files/code.

(b) This work represents our original work.

(c) We have not shared detailed analysis or detailed design results, computer code, or Simulink diagrams with any other student.

(d) We have not obtained or looked at lab reports from any other current or former student of ECE 481/ ECE 484 / MTE 484, and we have not let any other student access any part of our lab work.

(e) We have completely and unambiguously acknowledged and referenced all persons and aids used to help us with our work.

- ✓ **Student 1 Name:** Levi Rogalla
- ✓ **Student 1 Signature*:** 
- ✓ **Student 2 Name:** Alex Verić
- ✓ **Student 2 Signature*:** 
- ✓ **Lab Group #:** 57
- ✓ **Station #:** 3

(*) We require an actual signature: a typed name is not an acceptable substitute.

1 Question A

The controller must be designed in a way that allows the closed loop system to display the time-domain characteristics required, which are as follows: the step response steady-state tracking error is zero; he step response 2% settling time is no more than 0.25 seconds; the step response overshoot is no more than 5%; and the motor voltage does not exceed more than 6V in either direction.

The first requirement is already accounted for due to the nature of our plant. According to Internal Model Principle, in order for a closed loop system to have perfect error tracking, the open loop system (which consists of the plant and the controller) must contain a model of the reference signal in the LaPlace domain. In this situation, the given reference signal is a step input, which in the LaPlace domain is represented by $\frac{1}{s}$. The plant being used has already been characterized in the previous lab, and as can be seen in the following equation, already contains a model of the step input. Hence, it is not necessary to consider this in the controller design.

$$P(s) = \frac{K_1}{s(\tau s + 1)} \tag{1}$$

Where $K_1 = -1.8906$ and $\tau = 0.018107$.

The second two requirements, which regard the 2% settling time and the overshoot percentage, must be considered in the design of the controller. These values can be related to the poles of the closed loop system through the following sets of equations, which will determine the desired characteristic polynomial; using this information, as well as the previously derived plant model, the controller can be determined. The following equation shows the relation between the overshoot percentage and the damping ratio, ζ.

$$\zeta = -\frac{ln(OS\%)}{\sqrt{\pi^2 + [ln(OS\%)]}} \tag{2}$$

The following equation shows the relation between the 2% settling time, the damping ratio, ζ, and the natural frequency, ω_n.

$$\omega_n = \frac{4}{\zeta T_{s_{2\%}}} \tag{3}$$

The following equation is the characteristic polynomial of a second order, closed-loop system, in terms of the damping ratio, ζ, and the natural frequency, ω_n.

$$\pi(s) = s^2 + 2\zeta\omega_n s + \omega_n \tag{4}$$

Substituting $T_{s_{2\%}} = 0.25s$ and $OS\% = 0.05$ into the above equations give the following values for the damping ratio, ζ and the natural frequency, ω_n.

$$\zeta = 0.690, \omega_n = 23.185[\frac{1}{sec}] \tag{5}$$

Substituting these values into the characteristic polynomial gives the following equation:

$$\pi(s) = s^2 + 31.995s + 537.542 \tag{6}$$

Using the quadratic formula, as shown below, gives the conjugate pole pair that is required in the closed-loop system to have the required time-domain characteristics:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{7}$$

Where $a = 1$, $b = 31.995$, $c = 537.542$:

$$s_{1,2} = -15.998 \pm j16.782 \tag{8}$$

The above poles are the conjugate poles which will determine the behaviour of the closed-loop system. However, since the lab requires the controller to be at least first-order (and the derived plant is second order), the characteristic polynomial of the closed-loop system must be third-order. Thus, another pole is required. In order to maintain the desired behaviour already determined, this third pole must be non-dominant - this means it must lie on the real axis and be at least five times further from the imaginary axis than the conjugate pole pair. The reasoning for this is simple: the real part of a pole is what determines the exponential behaviour it contributes to the system. If the real value of the pole is significantly larger (and negative) than the other poles, it will decay much more quickly, resulting in its time-domain behaviour contribution being negligible.

Based on the above description, the third and non-dominant pole of the closed-loop system is $s_3 = -80$.

The third-order characteristic polynomial for the system is as follows:

$$\pi(s) = (s - s_1)(s - s_2)(s - s_3) \tag{9}$$

Substituting the three desired poles, s_1, s_2 and s_3:

$$\pi(s) = (s - (-15.998 + j16.782))(s - (-15.998 - j16.782))(s - (-80)) \tag{10}$$

Expanding and collecting like terms shows the full third order characteristic polynomial and its coefficients, which will be used briefly to determine those of the controller.

$$\pi(s) = s^3 + 111.996s^2 + 3097.252s + 45005.76 \tag{11}$$

The controller design will be done using the Sylvester matrix method, which will be described briefly. First, the coefficients of the characteristic polynomial must be gathered:

$$\alpha_3 = 1, \alpha_2 = 111.996, \alpha_1 = 3097.252, \alpha_0 = 45005.76 \tag{12}$$

Then, the plant must be made into the following form:

$$P(s) = \frac{b_2 s^2 + b_1 s - b_0}{s^2 + a_1 s + a_0} \tag{13}$$

Taking the plant derived in the previous lab, and transposing it to the above form gives the following:

$$P(s) = \frac{0s^2 + 0s - 104.413}{s^2 + 55.227s + 0} \tag{14}$$

And naming the coefficients gives:

$$b_2 = 0, b_1 = 0, b_0 = -104.413 a_1 = 55.227, a_0 = 0 \tag{15}$$

Assuming a proper, first-order controller of the following form:

$$C(s) = \frac{g_1 s + g_0}{f_1 s + f_0} \tag{16}$$

Now, the Sylvester matrix method will be described. It states that a matrix made up of the coefficients of the plant, called the Sylvester matrix, multiplied by an array made up of the coefficients of the controller is equal to an array made up by the coefficients of the resulting characteric polynomial. The following equation displays this:

$$\begin{bmatrix} 1 & 0 & b_2 & 0 \\ a_1 & 1 & b_1 & b_2 \\ a_0 & a_1 & b_0 & b_1 \\ 0 & a_0 & 0 & b_0 \end{bmatrix} \begin{bmatrix} f_1 \\ f_0 \\ g_1 \\ g_0 \end{bmatrix} = \begin{bmatrix} \alpha_3 \\ \alpha_2 \\ \alpha_1 \\ \alpha_0 \end{bmatrix} \tag{17}$$

Substituting all the gathered coefficients from the desired characteristic polynomial and the plant gives:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 55.227 & 1 & 0 & 0 \\ 0 & 55.227 & -104.413 & 0 \\ 0 & 0 & 0 & -104.413 \end{bmatrix} \begin{bmatrix} f_1 \\ f_0 \\ g_1 \\ g_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 111.996 \\ 3097.252 \\ 43005.760 \end{bmatrix} \tag{18}$$

Using simple system of equation rules allows for the calculation of each controller coefficient, as follows:

$$f_1 = 1, f_0 = 56.769, g_1 = 0.363, g_0 = -411.881 \tag{19}$$

Substituting these values into the controller model from equation 16 gives the following:

$$C(s) = \frac{0.363s - 411.881}{s + 56.769} \tag{20}$$

The above controller model was placed into the closed-loop simulation model in MatLab, with the previously derived plant. The reference signal was set to a step function which alternates between $-0.7rad$ and $0.7rad$, with a duty cycle of 50% and a period of 2 seconds, as instructed. Below is the model, followed by the time-domain response.

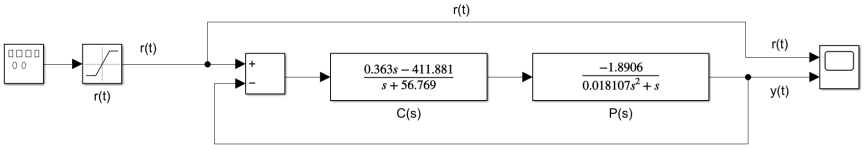


Figure 1: Model of closed-loop system.

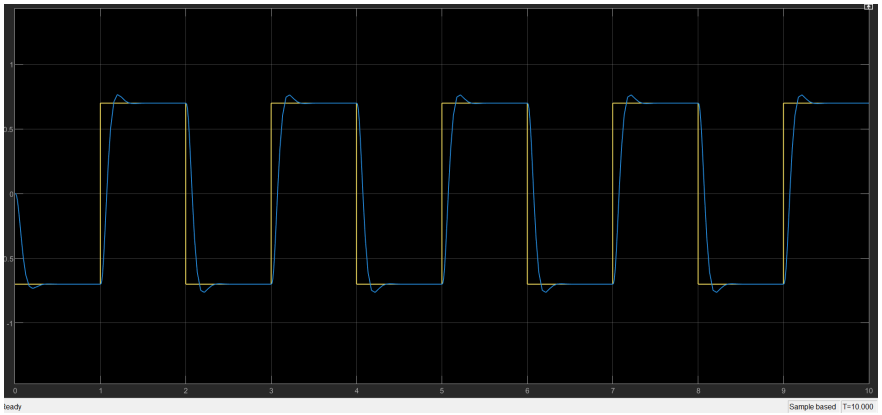


Figure 2: Time-domain response of closed-loop system to reference signal.

From figure 1, it is clear that the steady state error is equal to zero. In order to determine if the controller meets the other two requirements, the peak value and the settling time of the time-domain response signal must be measured.

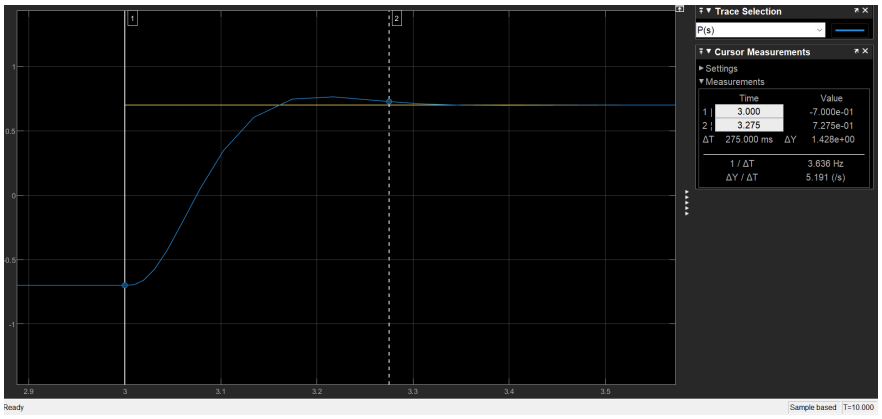


Figure 3: Measurement of 2% settling time

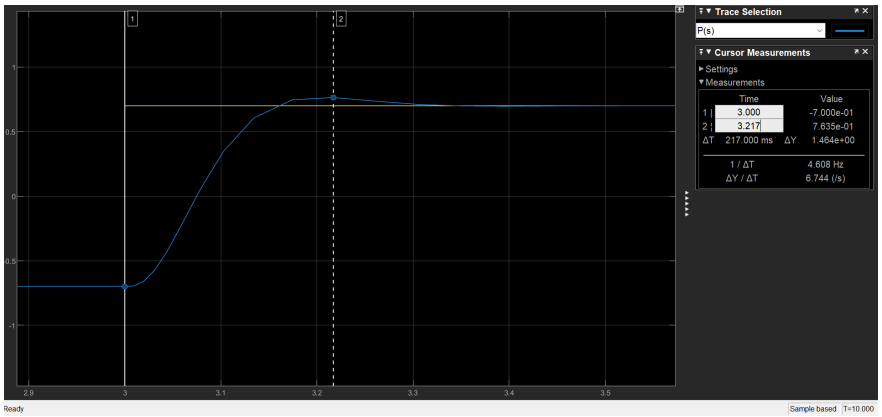


Figure 4: Measurement of overshoot percentage

The 2% settling time is the amount of time after the step response that is required for the

system output to reach 2% of the reference signal. Since the reference signal goes from -0.7 to 0.7, the total value of the step function is 1.4. Therefore, the 2% settling time occurs when the signal reaches 102% of 1.4, which is 1.428. The measurement in figure 1 shows that this occurs 0.275s after the unit step, which is slower than the required 0.25s.

The overshoot percentage, which is required to be no more than 5%, is measured in figure 1. The peak difference is measured to be 1.464; 105% of the unit step value would be 1.47, and so the overshoot percentage of this controller is within the acceptable range.

Since the 2% settling time is dependent on the natural frequency, ω_n, this would need to be modified in order to prevent the settling time from going above its limit. However, this would be time consuming, as it would require redoing nearly the entire process as described above. `systune`, a tool in MatLab, is a much more efficient way of tuning the existing controller to fit within the required settling time. The newly tuned controller has the following values, and its response to a step input can be seen in figure 1.

$$C(s) = \frac{-2.452s - 135.4}{s + 24.95} \tag{21}$$

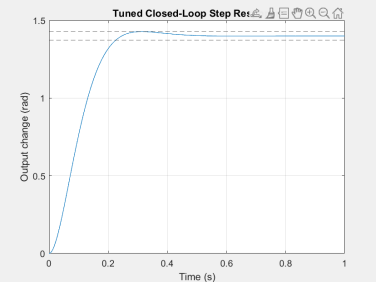


Figure 5: Tuned controller performance to step input.

The tuned controller has a 2% settling time of $0.2252s$ and an overshoot percentage of exactly 2.00%, both of which fit the required time-domain performance. The new controller is also shown to have a voltage within $\pm 6V$, as can be seen by the signal output by C(s) in figure 1.

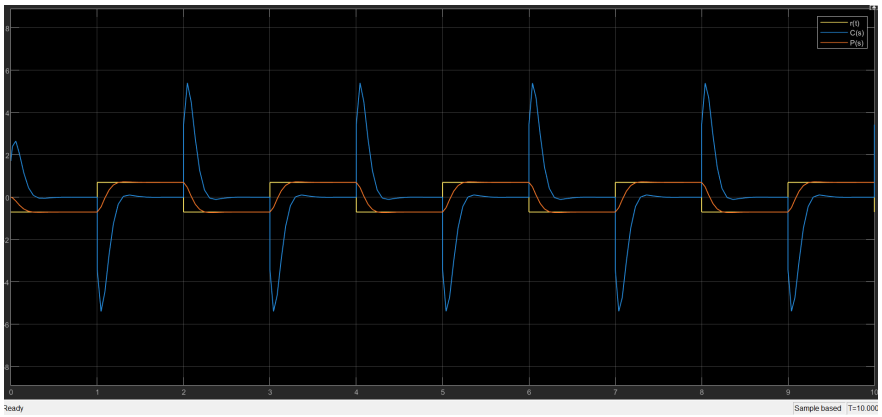


Figure 6: Motor voltage shown to be within $\pm 6V$ when using the tuned controller.

2 Question B

The discretization of a controller is necessary to allow a microcontroller, such as the Arduino Mega, to run the controller digitally. Software controllers cannot run in continuous time, and so have to sample the continuous time values, therefore making the system discrete. While a continuous controller can use derivatives to transform between LaPlace and time domains, discrete systems are non-continuous and therefore can not utilize derivatives. This results in discretization being necessary, and this can be done using one of two methods: bilinear transformation or step-invariant transformation. Bilinear transformation guarantees the stability of an already stable continuous controller, and is better at maintaining the time-domain behaviour of a continuous controller, while step-invariant transformation does not guarantee stability, but better maintains frequency domain responses. Since time-domain characteristics are those which are required, and stability is paramount in this system, bilinear transformation will be used to discretize the continuous controller.

Bilinear transformation involves replacing the Laplace domain variable s with the discrete domain variable z, using the following equation:

$$s = \frac{2}{T}\frac{z-1}{z+1} \tag{22}$$

Where T is the sampling period, which has been chosen to be 0.0005, making the sampling frequency equal to $2kHz$. Analyzing the time-domain step response, there are a few periods in which fast response is important, such as the peak time and the settling time, which are required characteristics that must be maintained during discretization. At these points, the controller must react quickly to limit the overshoot around the peak time and to reduce oscillations around the 2% settling time. Since the time constant which governs these values is 0.0625, the sampling time period was chosen to be roughly one hundred times smaller to allow the controller to react appropriately.

(Note: 200 Hz was initially used for the sampling frequency, but it was experimentally determined that this was too slow of a response to obtain the desired time-domain characteristics; for this reason, the frequency was increased by a factor of ten.)

Substituting s using equation 22 into the equation 16 for the continuous controller gives a new, discrete controller formula that should replicate its time-domain behaviour:

$$C[z] = \frac{(2g_1 + Tg_0)z + (Tg_0 - 2g_1)}{(2f_1 + Tf_0)z + (Tf_0 - 2f_1)} \tag{23}$$

Substituting the new coefficients of the tuned controller, as seen in equation 21, as well as the chosen sampling period, T, gives the following:

$$C[z] = \frac{-4.9717z + 4.8363}{2.012475z - 1.987525} \tag{24}$$

The discrete controller was then input into MatLab, where it was compared with the continuous controller step response.

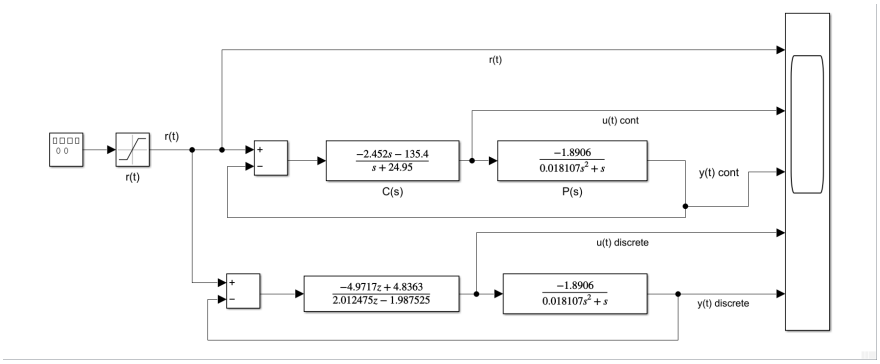


Figure 7: Discrete controller model in MatLab

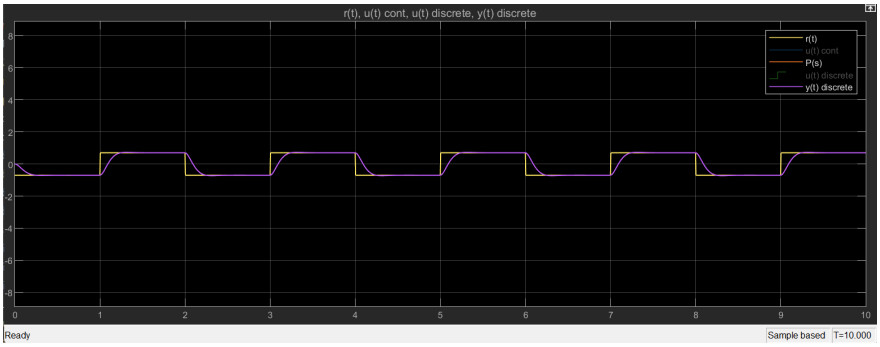


Figure 8: Discrete and continuous models step response compared. As can be seen, there is little to no visible difference between the time-domain characteristics of the two models.

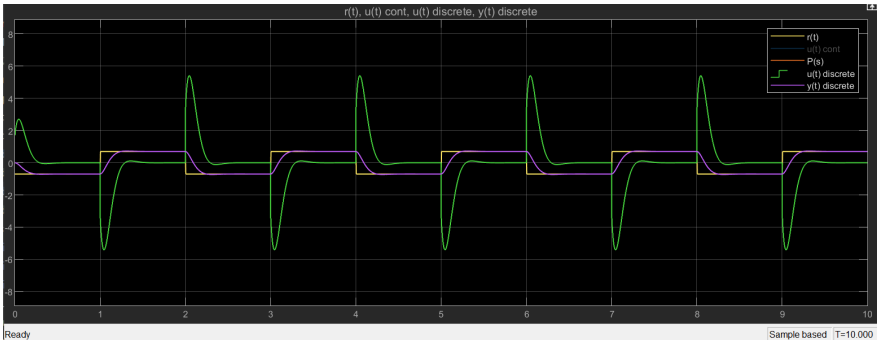


Figure 9: Motor voltage when using the discrete controller; it does not encroach $\pm 6V$

3 Question C

Now that the controller has been discretized, it can now be used to run the control system on the Arduino, which works digitally and in fixed sample intervals. However, there is one last step before the Arduino can use the controller: it must be transformed from the Z-domain into the sampling domain, which, unlike its counterpart, the time-domain, computes in fixed time intervals rather than continuous time. In order to do this, the controller must first be represented in terms of the signals which act as its input and output, as seen below:

$$C[z] = \frac{U[z]}{E[z]} = \frac{-4.9717z + 4.8363}{2.012475z - 1.987525} \tag{25}$$

Where U[z] is the discrete output of the controller, and E[z] is the discrete input. This equation must be rearranged as follows, to obtain a value for the output, based on the input:

$$U[z] = \frac{-4.9717z + 4.8363}{2.012475z - 1.987525} E[k] \tag{26}$$

Applying the inverse Z-transform:

$$u[k] = 0.987602u[k-1] - 2.470441e[k] + 2.403160e[k-1]. \tag{27}$$

Where k is an arbitrary sample, similar to how t is an arbitrary continuous unit of time, in the time-domain. The above equation was implemented in the Arduino code, along with its appropriate sampling time of $0.0005s$. The entire system was then tested with the a square wave input signal at 50% duty cycle and with a period of $2s$. The system time response is shown below in figure 3.

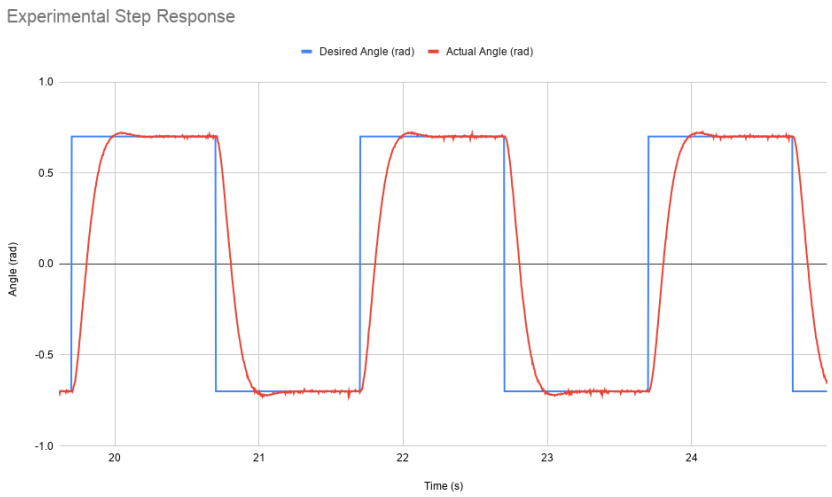


Figure 10: Step response of the motor

In order to ensure the time-domain characteristics required for this system are reached, figure 3 must be further inspected. The 2% settling time is reached almost exactly $0.25s$ after the step input is triggered. Additionally, the peak of the motor position is $0.72087rad$, which is roughly 2% of the unit step value; this meets the required maximum overshoot percentage of 5%. See below the exact values of the first recorded time-domain response from figure 3:

$$\begin{aligned} 2\%\text{ range} &= (0.672, 0.728), \\ T_{\text{signal}} &= 19.7017315\,\text{s}, \\ T_{s,\,2\%} &= 19.951723\,\text{s}, \\ T_{s,\,2\%} - T_{\text{signal}} &= 0.2499915\,\text{s}. \end{aligned}$$

Using the following equations to obtain $OS\%$:

$$OS\% = \frac{u(t)_{max} - r(t)_{max}}{|r(t)|} * 100\% \tag{28}$$

The following OS% can be obtained:

$$OS\% = 1.43928\% \tag{29}$$

The figure below shows the motor voltage as well, which never exceeds $\pm 6V$:

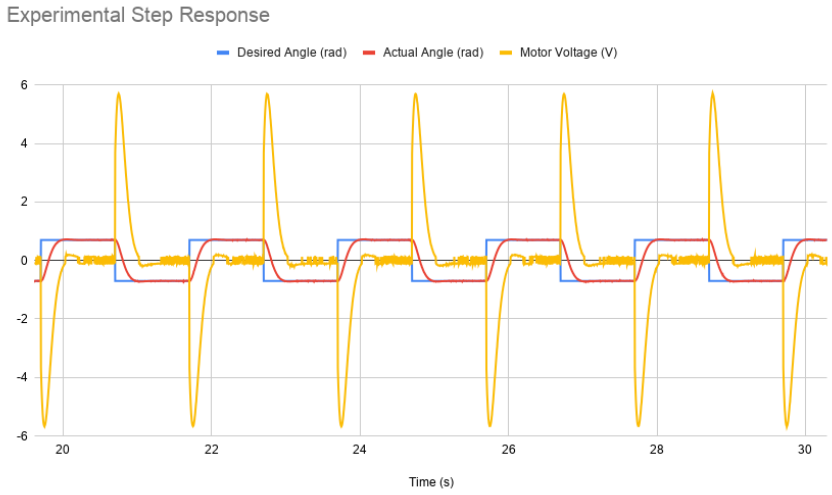


Figure 11: Step response and voltage of motor.

There are several differences between the performance of the simulated system and the real system when using the discretized controller. The 2% settling time of the real system is slightly longer than the simulated one, which can likely be attributed to the time delay introduced by the controller running on real hardware rather than inside a simulation. This presents itself in the form of a very slight delay between the unit step and the beginning of the motor movement, as can be seen in figure 3, which doesn't exist in the simulated system, in figure 1

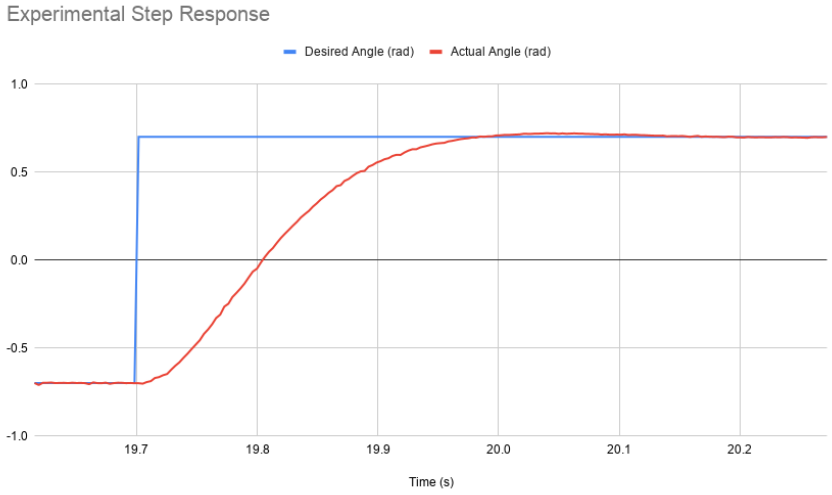


Figure 12: Zoomed in chart of the motor step response.

The experimental overshoot percentage is smaller than the simulated system. This was expected, as the forces introduced in a real system, such as internal friction or air resistance, increase the damping ratio of the system, resulting a smaller overshoot. The motor voltage of the system was seen to be higher than that of the simulated system, which was, once again, expected behaviour. The constant voltage added to the motor voltage by the Arduino code was one culprit of this higher motor voltage, as was the controller having to add more voltage as a result of the aforementioned internal friction and air resistance which do not exist in the simulated system.

4 Question D